Lab 9: Secure Access Control with a Finite State Machine

Module	ET4047
Date	08/11/2026
Lab No.	Lab 9
Author	Luke Griffin

Build a **digital security lock**: enter a 3-button code, see the result on the LCD and LEDs, and lock the system after three wrong attempts. Then attack your own lock and make it stronger.

By the end of this lab you can:

- read buttons reliably with `INPUT_PULLUP` and **debouncing**;
- write an embedded program as a **finite state machine** (FSM);
- time events with `millis()` instead of `delay()`;
- spot common **security weaknesses** in an embedded device and fix some of them.

1 Set-up

Item	Qty	Notes
Arduino Nano or Nano Every (on breadboard / shield)	1	USB cable
Push buttons	3	B1, B2, B3: the keypad
LEDs + 220 Ω resistors	2	Green = granted, Red = denied
16×2 LCD with I2C backpack, **or** Adafruit RGB LCD shield	1	`lab_lcd.h` finds either one
Slide switches (or jumper wires to GND)	3	Extension only

© 2026 by University of Limerick. Author: Luke Griffin

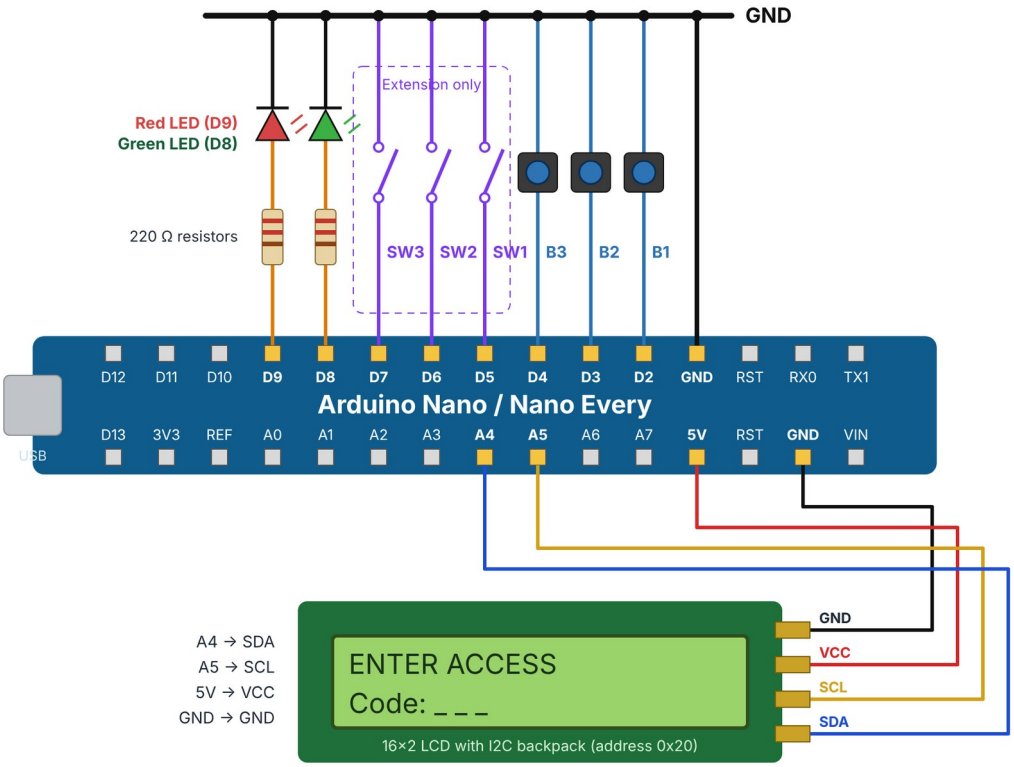


Figure 1: Wiring. Every button and switch goes from its pin to GND. LEDs go pin → 220 Ω → LED → GND.

Part	Pin	Part	Pin
B1 / B2 / B3	D2 / D3 / D4	Green LED	D8
SW1 / SW2 / SW3	D5 / D6 / D7	Red LED	D9
LCD SDA	A4	LCD SCL	A5
LCD VCC	5V	LCD GND	GND

Software

1. Arduino IDE → **Library Manager**: install **LiquidCrystal I2C** (Frank de Brabander) and **Adafruit RGB LCD Shield Library**.
2. Download the code from the lab repository: `Secure_Access_Control/Code/`. Every task has its own folder.
3. Each sketch needs the **lab_lcd.h** tab. Tasks 3, 4, 5 also need **buttons.h**. (Both files are already in each task folder.)

> **Hint**
>
> Board menu: **Arduino Nano** (Processor: ATmega328P, or *Old Bootloader* if upload fails) or **Arduino Nano Every**.
>
> To add a tab yourself: click ⋯ / ▾ at the right of the tabs → **New Tab** → type the file name → paste the code.

© 2026 by University of Limerick. Author: Luke Griffin

2 Lab tasks

Each task gives you the full code. Upload it, test it, then answer the questions. Mark items **(record)** in your report (photo or Serial Monitor output).

Task 1: Hardware test (15 min)

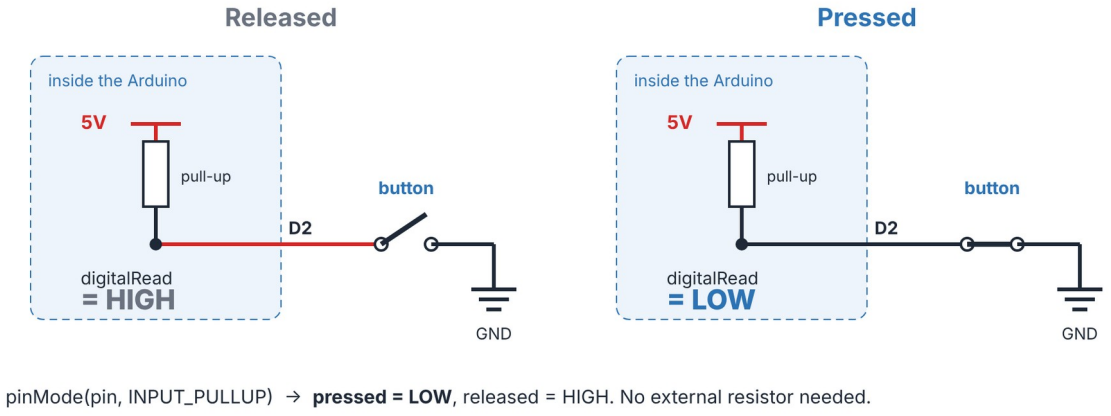


Figure 2: INPUT_PULLUP: the pin reads HIGH when released and LOW when pressed.

1. Wire the circuit as in Figure 1.
2. Upload **Task1_IO_Test**.
3. Press each button and flip each switch. The LCD must match Figure 3. B1 lights green, B2 lights red. **(record)** a photo.

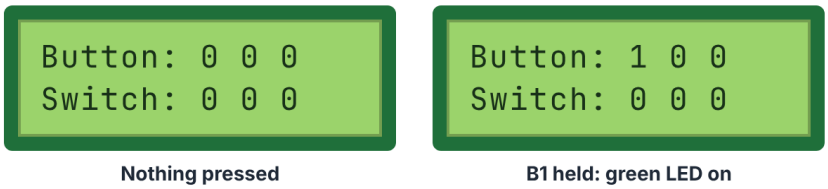


Figure 3: Task 1 expected LCD.

Task1_IO_Test.ino

```
// ET4047 Week 10 - Secure Access Control
// Task 1: Hardware test
// Shows the state of every button and switch on the LCD.
// Button 1 lights the green LED, Button 2 lights the red LED.

#include "lab_lcd.h"

LabLCD lcd(16, 2);

const byte BUTTON_PINS[3] = {2, 3, 4};   // B1, B2, B3
const byte SWITCH_PINS[3] = {5, 6, 7};   // SW1, SW2, SW3 (Extension only)
const byte LED_GREEN = 8;
const byte LED_RED   = 9;

// INPUT_PULLUP: the pin reads LOW when the button is pressed
bool isPressed(byte pin) {
  return digitalRead(pin) == LOW;
}
```

```
void setup() {
  Serial.begin(9600);

  for (byte i = 0; i < 3; i++) {
    pinMode(BUTTON_PINS[i], INPUT_PULLUP);
    pinMode(SWITCH_PINS[i], INPUT_PULLUP);
  }
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_RED, OUTPUT);

  lcd.init();
  lcd.backlight();
  Serial.print("LCD: ");
  Serial.println(lcd.name());
}

void loop() {
  // Row 0: buttons (1 = pressed)
  lcd.setCursor(0, 0);
  lcd.print("Button: ");
  for (byte i = 0; i < 3; i++) {
    lcd.print(isPressed(BUTTON_PINS[i]) ? '1' : '0');
    lcd.print(' ');
  }

  // Row 1: switches (1 = ON, connected to GND)
  lcd.setCursor(0, 1);
  lcd.print("Switch: ");
  for (byte i = 0; i < 3; i++) {
    lcd.print(isPressed(SWITCH_PINS[i]) ? '1' : '0');
    lcd.print(' ');
  }

  // LEDs follow Button 1 and Button 2
  digitalWrite(LED_GREEN, isPressed(BUTTON_PINS[0]) ? HIGH : LOW);
  digitalWrite(LED_RED,   isPressed(BUTTON_PINS[1]) ? HIGH : LOW);

  delay(50);   // a short delay is fine in a test sketch
}
```

If a digit never changes, check that wire. If the LCD is blank, check SDA/SCL and turn the contrast screw on the backpack.

© 2026 by University of Limerick. Author: Luke Griffin

Task 2: Debouncing and code entry (20 min)

A mechanical button **bounces**: one press gives several fast HIGH/LOW edges. Without debouncing, one press can be counted as two or three.

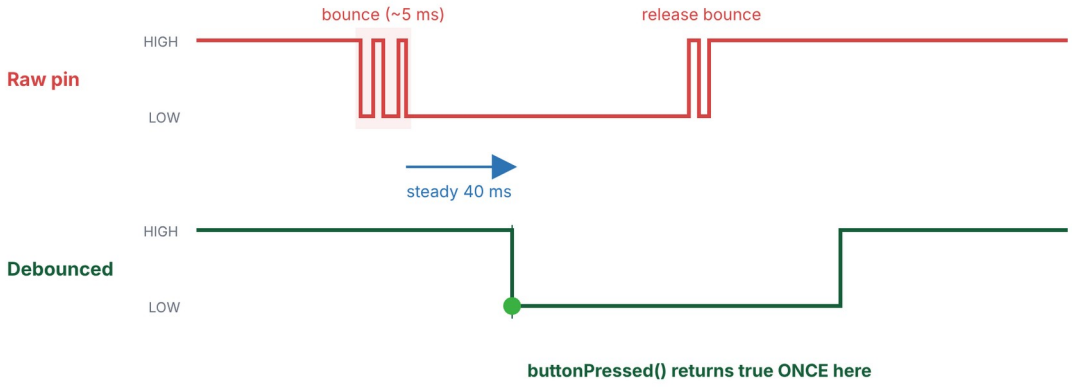


Figure 4: Bounce. The press is accepted only after the input has been steady for 40 ms.

Key code: buttonPressed() (full sketch: Appendix A1)

```
// Returns true ONCE per press (on the HIGH -> LOW edge)
bool buttonPressed(byte i) {
  bool reading = digitalRead(buttons[i].pin);

  if (reading != buttons[i].lastReading) {   // input moved: maybe a bounce
    if (reading == LOW) rawEdges++;
    buttons[i].lastReading = reading;
    buttons[i].lastChange = millis();
  }

  // Accept the new state only after it has been steady for DEBOUNCE_MS
  if (millis() - buttons[i].lastChange >= DEBOUNCE_MS &&
      reading != buttons[i].stableState) {
    buttons[i].stableState = reading;
    if (reading == LOW) {
      cleanPresses++;
      return true;
    }
  }
  return false;
}
```

1. Upload **Task2_Debounce**. Enter **1 3 2**. The LCD shows the code and two counters.
2. Press each button ten times. **(record)** `raw` and `clean`.
3. Set `DEBOUNCE_MS = 0`, upload again and repeat. **(record)** both counters again.

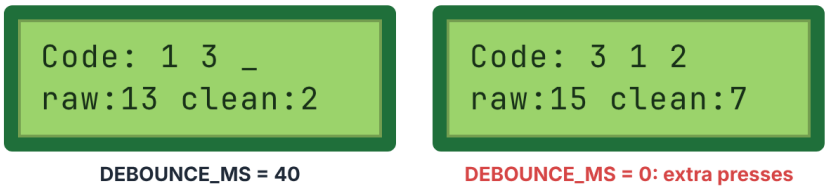


Figure 5: raw counts every edge; clean counts accepted presses.

DEBOUNCE_MS	raw	clean	Presses you made
40			30
0			30

Think about it

Why is `raw` bigger than `clean`? What goes wrong in a door lock if bounces are counted as presses?

Why does the code check for the **HIGH → LOW** change, rather than just `digitalRead() == LOW`?

Your answer:

© 2026 by University of Limerick. Author: Luke Griffin

Task 3: The finite state machine (20 min)

The lock is always in **one state**. Each state has its own outputs, and events (a button, a timer) move it to the next state.

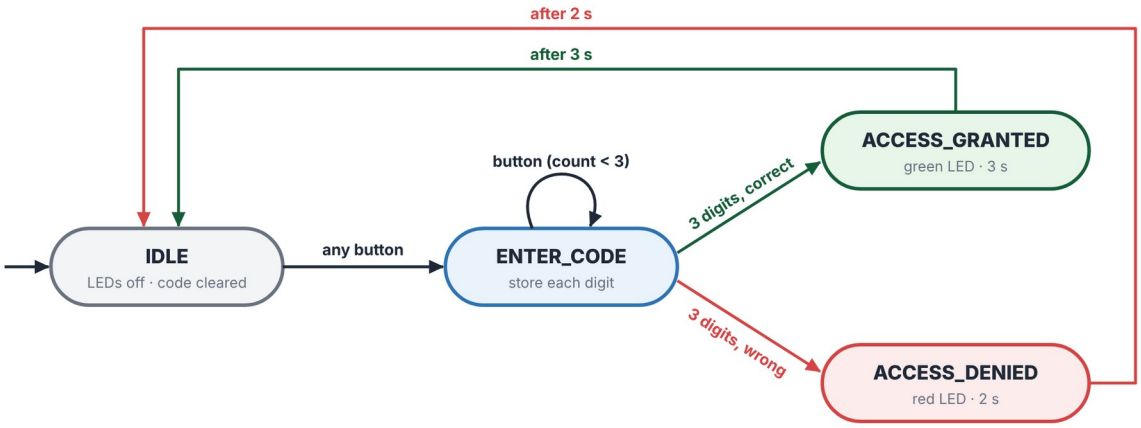


Figure 6: Task 3 FSM.

Key code: the states

```
enum State { IDLE, ENTER_CODE, ACCESS_GRANTED, ACCESS_DENIED };
State state;
unsigned long stateStart;        // millis() when we entered the state
```

Key code: loop() (full sketch: Appendix A3)

```
void loop() {
  byte b = readButtons();          // 0 = no new press
  unsigned long now = millis();

  switch (state) {
    case IDLE:
      if (b != 0) {                // first digit starts a new attempt
        entered[count++] = b;
        enterState(ENTER_CODE);
      }
      break;

    case ENTER_CODE:
      if (b != 0) {
        entered[count++] = b;
        showCode();
        if (count == CODE_LENGTH) {
          enterState(codeIsCorrect() ? ACCESS_GRANTED : ACCESS_DENIED);
        }
      }
      break;

    case ACCESS_GRANTED:
      if (now - stateStart >= GRANTED_MS) enterState(IDLE);
      break;

    case ACCESS_DENIED:
      if (now - stateStart >= DENIED_MS) enterState(IDLE);
      break;
  }
}
```

1. Upload **Task3_FSM** (with the **lab_lcd.h** and **buttons.h** tabs).
2. Open the Serial Monitor (9600 baud) and run the tests below. **(record)** the output.

Test	Input	Expected	✔
1	1 → 3 → 2	ACCESS GRANTED, green LED, back to IDLE after 3 s	
2	1 → 2 → 3	ACCESS DENIED, red LED, back to IDLE after 2 s	
3	Change `PASSWORD` to {2, 2, 1}	Only 2 → 2 → 1 is accepted	

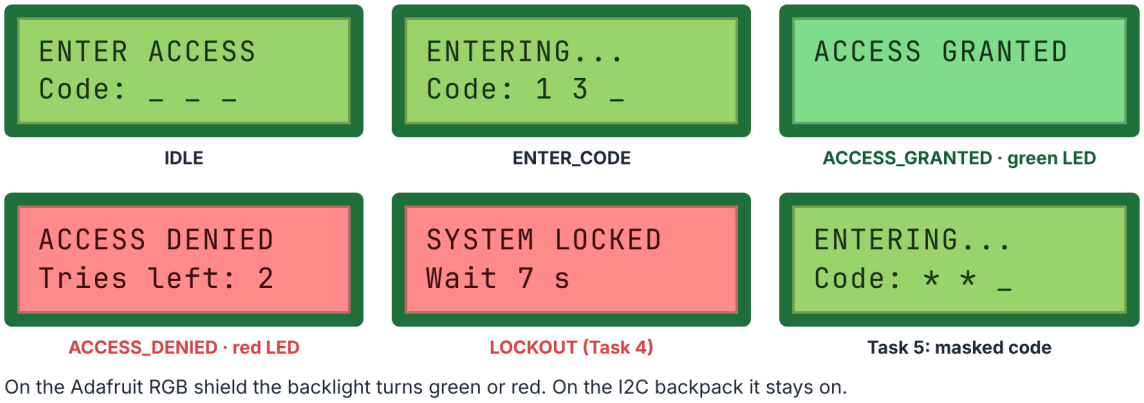


Figure 7: LCD in each state.

Think about it

`enterState()` sets the LEDs and LCD **once**, when the state changes. Why is that better than updating them on every pass of `loop()`?

Which state is the program in while you are typing the 2nd digit?

Your answer:

Task 4: Attempt counter and lockout (20 min)

Add a **LOCKOUT** state. Three wrong codes (or taking longer than 10 s) lock the keypad for 10 s.

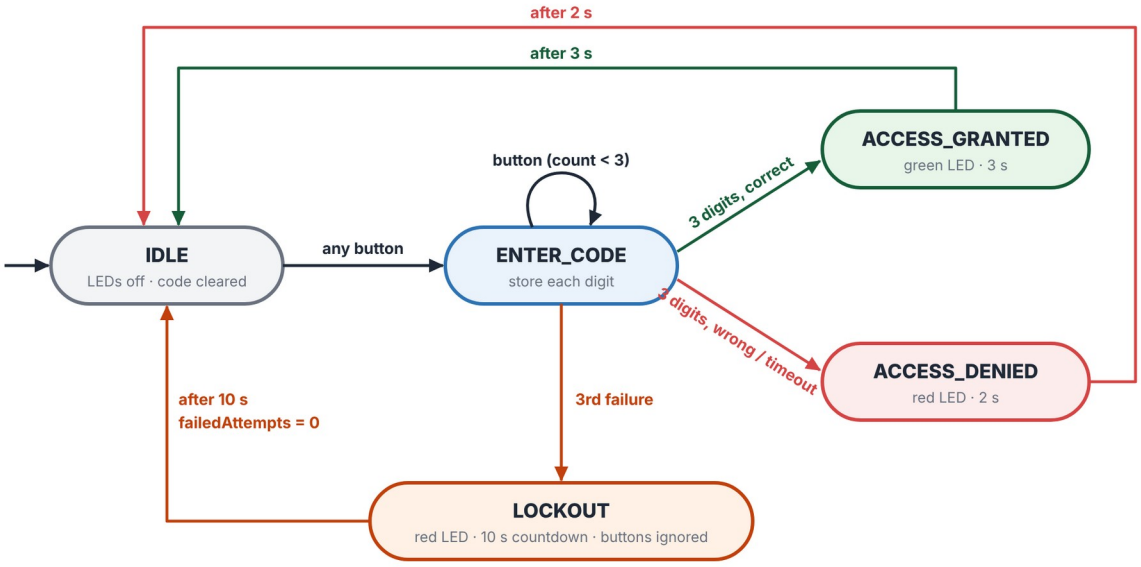


Figure 8: Task 4 FSM: the new LOCKOUT state.

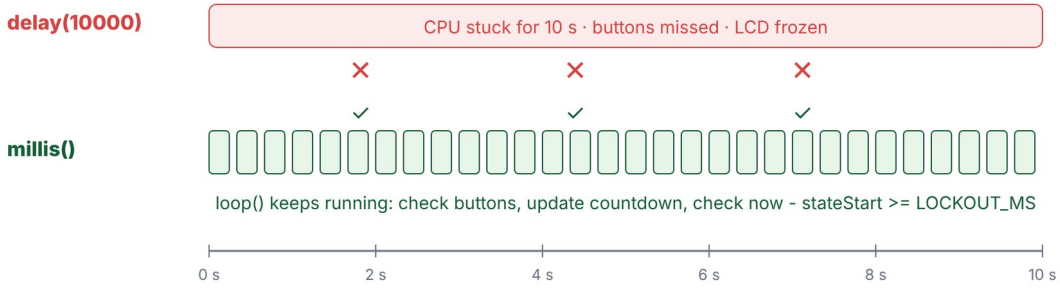


Figure 9: Why millis(): during the lockout the loop keeps running, so the countdown updates and buttons are still read.

Key code: failedAttempt()

```
// Called when a code is wrong OR the entry time runs out
void failedAttempt() {
  failedAttempts++;
  if (failedAttempts >= MAX_ATTEMPTS) enterState(LOCKOUT);
  else                                enterState(ACCESS_DENIED);
}
```

Key code: the LOCKOUT state (full sketch: Appendix A4)

```
    case LOCKOUT:
      // Buttons are read but ignored: nothing can be entered while locked
      if (elapsed >= LOCKOUT_MS) {
        failedAttempts = 0;
        enterState(IDLE);
      } else {
        int secondsLeft = (LOCKOUT_MS - elapsed + 999) / 1000;
        if (secondsLeft != lastSecondsShown) {      // redraw once a second
          lastSecondsShown = secondsLeft;
          lcd.setCursor(0, 1);
          lcd.print("Wait ");
          lcd.print(secondsLeft);
          lcd.print(" s   ");
        }
      }
      break;
```

1. Upload **Task4_Lockout** and run the tests. **(record)** a photo of the LOCKOUT screen.

Test	Input	Expected	✔
1	Wrong code 3 times	Tries left: 2, then 1, then SYSTEM LOCKED	
2	Press buttons while locked	Ignored. Countdown keeps going	
3	Wait 10 s	Back to ENTER ACCESS	
4	Press one button, then wait	After 10 s: ACCESS DENIED (timeout)	
5	1 → 3 → 2	ACCESS GRANTED; counter goes back to 0	

Think about it

What would happen to the countdown and the buttons if the lockout used `delay(10000)`?

Why `now - stateStart >= LOCKOUT_MS` and not `millis() == stateStart + LOCKOUT_MS`?

Your answer:

© 2026 by University of Limerick. Author: Luke Griffin

Task 5: Cybersecurity challenge (20 min)

Now be the **attacker**. Use your Task 4 lock and find its weaknesses.

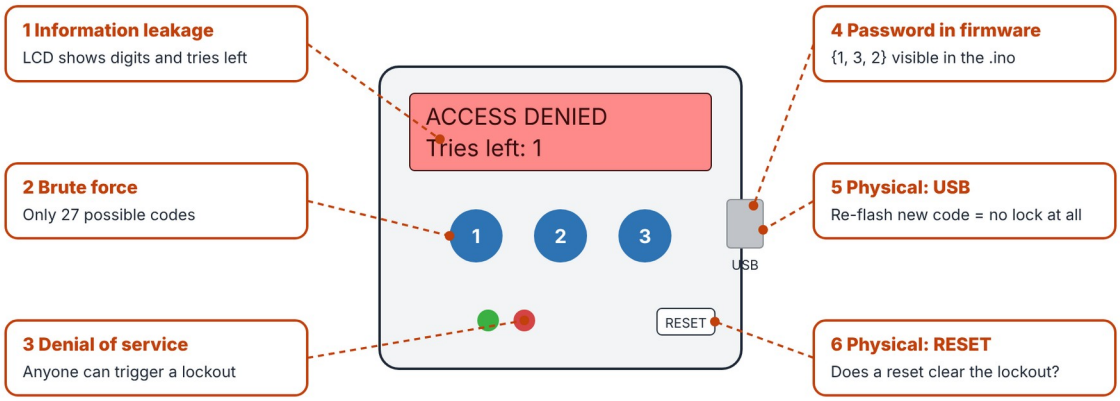


Figure 10: Six ways to attack the Task 4 lock.

#	Attack	Try it / answer it	Your answer
1	Information leakage	What can someone standing behind you learn from the LCD?	
2	Brute force	How many possible codes are there? With 4 digits?	
3	Denial of service	Can a stranger stop *you* from getting in?	
4	Password in firmware	Where is the password stored? Who can read it?	
5	Physical: USB	What stops someone uploading a sketch with no lock?	
6	Physical: RESET	Lock the system, then press RESET. What happens?	



Figure 11: A lockout slows brute force down but does not stop it: the code is still too short.

Upload **Task5_Secure**. It fixes some of these weaknesses:

Fix	What changed	Attack
[1]	Code shown as `* * _`	1
[2]	Denied screen gives no "tries left"	1
[3]	Progressive lockout: 10 s, 20 s, 40 s … 160 s	2
[4]	Failed attempts saved in **EEPROM**: RESET or power off does not clear a lockout	6
[5]	Every digit is always compared (no early exit)	timing

Key code: [4] EEPROM (full sketch: Appendix A5)

```
void saveCounters() {
  EEPROM.update(EE_FAILED, failedAttempts);   // update() only writes if changed
  EEPROM.update(EE_LOCKOUTS, lockoutLevel);
}

void loadCounters() {
  if (EEPROM.read(EE_MAGIC) != MAGIC) {       // first run: start clean
    EEPROM.update(EE_MAGIC, MAGIC);
    failedAttempts = 0;
    lockoutLevel = 0;
    saveCounters();
  } else {
    failedAttempts = EEPROM.read(EE_FAILED);
    lockoutLevel = min(EEPROM.read(EE_LOCKOUTS), MAX_LOCKOUT_LEVEL);
```

1. Repeat the Task 4 tests. Then lock the system and press **RESET**: it must stay locked. **(record)**
2. Lock it a 2nd time. The countdown should start at **20 s**.
3. Choose **one** more improvement and add it yourself, e.g. a 4-digit code, or a 1 s delay after every wrong code.

Important

EEPROM is **not encryption**. It keeps data after power off, but anyone with the board can read it.

Attacks 4 and 5 are still open: with USB access, the firmware can be read or replaced.

Think about it

Which attacks are still possible with Task5_Secure? Which improvement did you add, and which attack does it slow down?

Your answer:

© 2026 by University of Limerick. Author: Luke Griffin

Extension: Operating modes with switches (optional)

Add SW1–SW3 (Figure 1). A jumper wire from the pin to GND works as a switch (ON = connected to GND).

Switch	ON means	Code
SW1 (D5)	System enabled. OFF → SYSTEM OFF, keypad ignored	–
SW2 (D6)	Enhanced security: only 5 s to enter the code	1-3-2
SW3 (D7)	Maintenance: needs a separate code and clears the lockout level	3-2-1

1. Upload **Extension_Switches** (folder in the repository). Test each mode.
2. Turn SW1 OFF during a lockout. The lockout must still finish first. Why is that important?

Your answer:

3 Demonstrate and submit

#	Show your demonstrator	✓
1	All buttons, LEDs and LCD working (Task 1)	
2	Correct code → ACCESS GRANTED	
3	Wrong code → ACCESS DENIED	
4	Three wrong codes → LOCKOUT, back to IDLE after 10 s	
5	At least one security improvement (Task 5)	

Submit: your `.ino` files, a photo of your circuit, the answers in this document, and two sentences on the main remaining weakness of your lock.

Appendix A: Full code listings

The same files are in the repository. **lab_lcd.h** is a helper for both LCD types: download it, you do not need to edit it.

© 2026 by University of Limerick. Author: Luke Griffin

A1 Task2_Debounce.ino

Task2_Debounce.ino

```
// ET4047 Week 10 - Secure Access Control
// Task 2: Debounced buttons and code entry
// Each press is counted once. The LCD shows the code typed so far
// and compares raw edges (with bounce) against clean presses.

#include "lab_lcd.h"

LabLCD lcd(16, 2);

const byte BUTTON_PINS[3] = {2, 3, 4};
const unsigned long DEBOUNCE_MS = 40;    // try 0 and compare the counts

// ---------- Debounced button ----------
struct Button {
  byte pin;
  bool stableState;            // last accepted state
  bool lastReading;            // last raw reading
  unsigned long lastChange;    // time of the last raw change
};

Button buttons[3];
unsigned int rawEdges = 0;     // every HIGH->LOW edge, including bounces
unsigned int cleanPresses = 0; // presses accepted after debouncing

// Returns true ONCE per press (on the HIGH -> LOW edge)
bool buttonPressed(byte i) {
  bool reading = digitalRead(buttons[i].pin);

  if (reading != buttons[i].lastReading) {   // input moved: maybe a bounce
    if (reading == LOW) rawEdges++;
    buttons[i].lastReading = reading;
    buttons[i].lastChange = millis();
  }

  // Accept the new state only after it has been steady for DEBOUNCE_MS
  if (millis() - buttons[i].lastChange >= DEBOUNCE_MS &&
      reading != buttons[i].stableState) {
    buttons[i].stableState = reading;
    if (reading == LOW) {
      cleanPresses++;
      return true;
    }
  }
  return false;
}

// Returns 1, 2 or 3 for a new press, or 0 for no press
byte readButtons() {
  for (byte i = 0; i < 3; i++) {
    if (buttonPressed(i)) return i + 1;
  }
  return 0;
}

// ---------- Code entry ----------
const byte CODE_LENGTH = 3;
byte entered[CODE_LENGTH];
byte count = 0;

void clearCode() {
  count = 0;
}
```

```
void showScreen() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Code: ");
  for (byte i = 0; i < CODE_LENGTH; i++) {
    if (i < count) lcd.print(entered[i]);
    else           lcd.print('_');
    lcd.print(' ');
  }
  lcd.setCursor(0, 1);
  lcd.print("raw:");
  lcd.print(rawEdges);
  lcd.print(" clean:");
  lcd.print(cleanPresses);
}

void setup() {
  Serial.begin(9600);
  for (byte i = 0; i < 3; i++) {
    buttons[i].pin = BUTTON_PINS[i];
    pinMode(BUTTON_PINS[i], INPUT_PULLUP);
    buttons[i].stableState = HIGH;
    buttons[i].lastReading = HIGH;
    buttons[i].lastChange = 0;
  }
  lcd.init();
  lcd.backlight();
  showScreen();
}

void loop() {
  byte b = readButtons();
  if (b != 0) {
    if (count == CODE_LENGTH) clearCode();   // 4th press starts a new code
    entered[count++] = b;
    Serial.print("Button ");
    Serial.println(b);
    showScreen();
  }
}
```

© 2026 by University of Limerick. Author: Luke Griffin

A2 buttons.h (Tasks 3, 4, 5)

buttons.h

```
#ifndef BUTTONS_H
#define BUTTONS_H

// ------------------------------------------------------------
// Debounced push buttons (the Task 2 code, packaged for reuse)
//
//   initButtons();          // in setup()
//   byte b = readButtons(); // in loop(): 1, 2, 3 = new press, 0 = none
//
// Buttons are wired pin -> button -> GND and use INPUT_PULLUP,
// so a pressed button reads LOW.
// ------------------------------------------------------------

#include <Arduino.h>

const byte BUTTON_PINS[3] = {2, 3, 4};   // B1, B2, B3
const unsigned long DEBOUNCE_MS = 40;

struct Button {
  byte pin;
  bool stableState;
  bool lastReading;
  unsigned long lastChange;
};

Button buttons[3];

void initButtons() {
  for (byte i = 0; i < 3; i++) {
    buttons[i].pin = BUTTON_PINS[i];
    pinMode(BUTTON_PINS[i], INPUT_PULLUP);
    buttons[i].stableState = HIGH;
    buttons[i].lastReading = HIGH;
    buttons[i].lastChange = 0;
  }
}

// True once per press (HIGH -> LOW after DEBOUNCE_MS of steady input)
bool buttonPressed(byte i) {
  bool reading = digitalRead(buttons[i].pin);
  if (reading != buttons[i].lastReading) {
    buttons[i].lastReading = reading;
    buttons[i].lastChange = millis();
  }
  if (millis() - buttons[i].lastChange >= DEBOUNCE_MS &&
      reading != buttons[i].stableState) {
    buttons[i].stableState = reading;
    return reading == LOW;
  }
  return false;
}

// 1, 2 or 3 for a new press, 0 for none
byte readButtons() {
  for (byte i = 0; i < 3; i++) {
    if (buttonPressed(i)) return i + 1;
  }
  return 0;
}

#endif // BUTTONS_H
```

A3 Task3_FSM.ino

Task3_FSM.ino

```
// ET4047 Week 10 - Secure Access Control
// Task 3: Finite State Machine
// IDLE -> ENTER_CODE -> ACCESS_GRANTED / ACCESS_DENIED -> IDLE
// Needs the lab_lcd.h and buttons.h tabs.

#include "lab_lcd.h"
#include "buttons.h"

LabLCD lcd(16, 2);

const byte LED_GREEN = 8;
const byte LED_RED   = 9;

const byte CODE_LENGTH = 3;
const byte PASSWORD[CODE_LENGTH] = {1, 3, 2};

const unsigned long GRANTED_MS = 3000;
const unsigned long DENIED_MS  = 2000;

// RGB shield backlight colours (any colour = "on" for the backpack LCD)
const byte RED = 0x1, GREEN = 0x2, WHITE = 0x7;

// ---------- FSM ----------
enum State { IDLE, ENTER_CODE, ACCESS_GRANTED, ACCESS_DENIED };
State state;
unsigned long stateStart;        // millis() when we entered the state

byte entered[CODE_LENGTH];
byte count = 0;

void showCode() {
  lcd.setCursor(0, 1);
  lcd.print("Code: ");
  for (byte i = 0; i < CODE_LENGTH; i++) {
    if (i < count) lcd.print(entered[i]);
    else           lcd.print('_');
    lcd.print(' ');
  }
}

bool codeIsCorrect() {
  for (byte i = 0; i < CODE_LENGTH; i++) {
    if (entered[i] != PASSWORD[i]) return false;
  }
  return true;
}

// Every state change goes through here: set outputs ONCE on entry
void enterState(State s) {
  state = s;
  stateStart = millis();
  lcd.clear();
  digitalWrite(LED_GREEN, LOW);
  digitalWrite(LED_RED, LOW);
  lcd.setBacklight(WHITE);

  switch (s) {
    case IDLE:
      count = 0;
      lcd.print("ENTER ACCESS");
      showCode();
      Serial.println("IDLE");
```

```
      break;
    case ENTER_CODE:
      lcd.print("ENTERING...");
      showCode();
      Serial.println("ENTER_CODE");
      break;
    case ACCESS_GRANTED:
      digitalWrite(LED_GREEN, HIGH);
      lcd.setBacklight(GREEN);
      lcd.print("ACCESS GRANTED");
      Serial.println("ACCESS_GRANTED");
      break;
    case ACCESS_DENIED:
      digitalWrite(LED_RED, HIGH);
      lcd.setBacklight(RED);
      lcd.print("ACCESS DENIED");
      Serial.println("ACCESS_DENIED");
      break;
  }
}

void setup() {
  Serial.begin(9600);
  initButtons();
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_RED, OUTPUT);
  lcd.init();
  lcd.backlight();
  enterState(IDLE);
}

void loop() {
  byte b = readButtons();          // 0 = no new press
  unsigned long now = millis();

  switch (state) {
    case IDLE:
      if (b != 0) {                // first digit starts a new attempt
        entered[count++] = b;
        enterState(ENTER_CODE);
      }
      break;

    case ENTER_CODE:
      if (b != 0) {
        entered[count++] = b;
        showCode();
        if (count == CODE_LENGTH) {
          enterState(codeIsCorrect() ? ACCESS_GRANTED : ACCESS_DENIED);
        }
      }
      break;

    case ACCESS_GRANTED:
      if (now - stateStart >= GRANTED_MS) enterState(IDLE);
      break;

    case ACCESS_DENIED:
      if (now - stateStart >= DENIED_MS) enterState(IDLE);
      break;
  }
}
```

© 2026 by University of Limerick. Author: Luke Griffin

A4 Task4_Lockout.ino

Task4_Lockout.ino

```
// ET4047 Week 10 - Secure Access Control
// Task 4: Authentication with attempt counter, lockout and timeout
// Adds the LOCKOUT state to the Task 3 FSM. All timing uses millis().
// Needs the lab_lcd.h and buttons.h tabs.

#include "lab_lcd.h"
#include "buttons.h"

LabLCD lcd(16, 2);

const byte LED_GREEN = 8;
const byte LED_RED   = 9;

const byte CODE_LENGTH = 3;
const byte PASSWORD[CODE_LENGTH] = {1, 3, 2};

const byte MAX_ATTEMPTS = 3;
const unsigned long GRANTED_MS = 3000;
const unsigned long DENIED_MS  = 2000;
const unsigned long ENTRY_MS   = 10000;   // finish the code within 10 s
const unsigned long LOCKOUT_MS = 10000;   // locked for 10 s

const byte RED = 0x1, GREEN = 0x2, WHITE = 0x7;

// ---------- FSM ----------
enum State { IDLE, ENTER_CODE, ACCESS_GRANTED, ACCESS_DENIED, LOCKOUT };
State state;
unsigned long stateStart;

byte entered[CODE_LENGTH];
byte count = 0;
byte failedAttempts = 0;
int lastSecondsShown = -1;       // so the countdown redraws once a second

void showCode() {
  lcd.setCursor(0, 1);
  lcd.print("Code: ");
  for (byte i = 0; i < CODE_LENGTH; i++) {
    if (i < count) lcd.print(entered[i]);
    else           lcd.print('_');
    lcd.print(' ');
  }
}

bool codeIsCorrect() {
  for (byte i = 0; i < CODE_LENGTH; i++) {
    if (entered[i] != PASSWORD[i]) return false;
  }
  return true;
}

void enterState(State s) {
  state = s;
  stateStart = millis();
  lcd.clear();
  digitalWrite(LED_GREEN, LOW);
  digitalWrite(LED_RED, LOW);
  lcd.setBacklight(WHITE);

  switch (s) {
    case IDLE:
      count = 0;
```

```
      lcd.print("ENTER ACCESS");
      showCode();
      break;
    case ENTER_CODE:
      lcd.print("ENTERING...");
      showCode();
      break;
    case ACCESS_GRANTED:
      digitalWrite(LED_GREEN, HIGH);
      lcd.setBacklight(GREEN);
      lcd.print("ACCESS GRANTED");
      break;
    case ACCESS_DENIED:
      digitalWrite(LED_RED, HIGH);
      lcd.setBacklight(RED);
      lcd.print("ACCESS DENIED");
      lcd.setCursor(0, 1);
      lcd.print("Tries left: ");
      lcd.print(MAX_ATTEMPTS - failedAttempts);
      break;
    case LOCKOUT:
      digitalWrite(LED_RED, HIGH);
      lcd.setBacklight(RED);
      lcd.print("SYSTEM LOCKED");
      lastSecondsShown = -1;
      break;
  }
  Serial.print("State -> ");
  Serial.println(s);
}

// Called when a code is wrong OR the entry time runs out
void failedAttempt() {
  failedAttempts++;
  if (failedAttempts >= MAX_ATTEMPTS) enterState(LOCKOUT);
  else                                enterState(ACCESS_DENIED);
}

void setup() {
  Serial.begin(9600);
  initButtons();
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_RED, OUTPUT);
  lcd.init();
  lcd.backlight();
  enterState(IDLE);
}

void loop() {
  byte b = readButtons();
  unsigned long now = millis();
  unsigned long elapsed = now - stateStart;

  switch (state) {
    case IDLE:
      if (b != 0) {
        entered[count++] = b;
        enterState(ENTER_CODE);
      }
      break;

    case ENTER_CODE:
      if (elapsed >= ENTRY_MS) {          // too slow: counts as a failure
        failedAttempt();
      } else if (b != 0) {
        entered[count++] = b;
        showCode();
        if (count == CODE_LENGTH) {
          if (codeIsCorrect()) {
```

```
            failedAttempts = 0;
            enterState(ACCESS_GRANTED);
          } else {
            failedAttempt();
          }
        }
      }
      break;

    case ACCESS_GRANTED:
      if (elapsed >= GRANTED_MS) enterState(IDLE);
      break;

    case ACCESS_DENIED:
      if (elapsed >= DENIED_MS) enterState(IDLE);
      break;

    case LOCKOUT:
      // Buttons are read but ignored: nothing can be entered while locked
      if (elapsed >= LOCKOUT_MS) {
        failedAttempts = 0;
        enterState(IDLE);
      } else {
        int secondsLeft = (LOCKOUT_MS - elapsed + 999) / 1000;
        if (secondsLeft != lastSecondsShown) {      // redraw once a second
          lastSecondsShown = secondsLeft;
          lcd.setCursor(0, 1);
          lcd.print("Wait ");
          lcd.print(secondsLeft);
          lcd.print(" s   ");
        }
      }
      break;
  }
}
```

© 2026 by University of Limerick. Author: Luke Griffin

A5 Task5_Secure.ino

Task5_Secure.ino

```
// ET4047 Week 10 - Secure Access Control
// Task 5: Security improvements to the Task 4 lock
//   [1] Code is masked with '*' (no shoulder surfing)
//   [2] Denied screen gives no hints (no "tries left")
//   [3] Progressive lockout: 10 s, 20 s, 40 s ... up to 160 s
//   [4] Failed attempts are kept in EEPROM, so RESET / power off
//       does not clear a lockout
//   [5] Every digit is always checked (no early exit)
// Needs the lab_lcd.h and buttons.h tabs.

#include <EEPROM.h>
#include "lab_lcd.h"
#include "buttons.h"

LabLCD lcd(16, 2);

const byte LED_GREEN = 8;
const byte LED_RED   = 9;

const byte CODE_LENGTH = 3;
const byte PASSWORD[CODE_LENGTH] = {1, 3, 2};

const byte MAX_ATTEMPTS = 3;
const unsigned long GRANTED_MS      = 3000;
const unsigned long DENIED_MS       = 2000;
const unsigned long ENTRY_MS        = 10000;
const unsigned long BASE_LOCKOUT_MS = 10000;
const byte MAX_LOCKOUT_LEVEL        = 4;      // 10 s * 2^4 = 160 s

const byte RED = 0x1, GREEN = 0x2, WHITE = 0x7;

// [4] EEPROM addresses
const int EE_MAGIC    = 0;    // 0xA5 once the EEPROM has been set up
const int EE_FAILED   = 1;    // failed attempts since last success
const int EE_LOCKOUTS = 2;    // how many lockouts in a row
const byte MAGIC = 0xA5;

enum State { IDLE, ENTER_CODE, ACCESS_GRANTED, ACCESS_DENIED, LOCKOUT };
State state;
unsigned long stateStart;
unsigned long lockoutMs;

byte entered[CODE_LENGTH];
byte count = 0;
byte failedAttempts = 0;
byte lockoutLevel = 0;
int lastSecondsShown = -1;

void saveCounters() {
  EEPROM.update(EE_FAILED, failedAttempts);   // update() only writes if changed
  EEPROM.update(EE_LOCKOUTS, lockoutLevel);
}

void loadCounters() {
  if (EEPROM.read(EE_MAGIC) != MAGIC) {       // first run: start clean
    EEPROM.update(EE_MAGIC, MAGIC);
    failedAttempts = 0;
    lockoutLevel = 0;
    saveCounters();
  } else {
    failedAttempts = EEPROM.read(EE_FAILED);
    lockoutLevel = min(EEPROM.read(EE_LOCKOUTS), MAX_LOCKOUT_LEVEL);
  }
```

```
}

void showCode() {
  lcd.setCursor(0, 1);
  lcd.print("Code: ");
  for (byte i = 0; i < CODE_LENGTH; i++) {
    lcd.print(i < count ? '*' : '_');          // [1] masked
    lcd.print(' ');
  }
}

// [5] Always compares every digit, so the time taken never depends
//     on how many digits were right
bool codeIsCorrect() {
  byte differences = 0;
  for (byte i = 0; i < CODE_LENGTH; i++) {
    differences |= entered[i] ^ PASSWORD[i];
  }
  return differences == 0;
}

void enterState(State s) {
  state = s;
  stateStart = millis();
  lcd.clear();
  digitalWrite(LED_GREEN, LOW);
  digitalWrite(LED_RED, LOW);
  lcd.setBacklight(WHITE);

  switch (s) {
    case IDLE:
      count = 0;
      lcd.print("ENTER ACCESS");
      showCode();
      break;
    case ENTER_CODE:
      lcd.print("ENTERING...");
      showCode();
      break;
    case ACCESS_GRANTED:
      digitalWrite(LED_GREEN, HIGH);
      lcd.setBacklight(GREEN);
      lcd.print("ACCESS GRANTED");
      break;
    case ACCESS_DENIED:
      digitalWrite(LED_RED, HIGH);
      lcd.setBacklight(RED);
      lcd.print("ACCESS DENIED");               // [2] nothing else
      break;
    case LOCKOUT:
      digitalWrite(LED_RED, HIGH);
      lcd.setBacklight(RED);
      lcd.print("SYSTEM LOCKED");
      lockoutMs = BASE_LOCKOUT_MS << lockoutLevel;   // [3] 10,20,40,80,160 s
      lastSecondsShown = -1;
      break;
  }
}

void failedAttempt() {
  failedAttempts++;
  saveCounters();                 // [4] saved BEFORE any lockout starts
  if (failedAttempts >= MAX_ATTEMPTS) enterState(LOCKOUT);
  else                                enterState(ACCESS_DENIED);
}

void setup() {
  Serial.begin(9600);
```

```
  initButtons();
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_RED, OUTPUT);
  lcd.init();
  lcd.backlight();
  loadCounters();

  // [4] Reset during a lockout? Start the lockout again.
  if (failedAttempts >= MAX_ATTEMPTS) enterState(LOCKOUT);
  else                                enterState(IDLE);
}

void loop() {
  byte b = readButtons();
  unsigned long elapsed = millis() - stateStart;

  switch (state) {
    case IDLE:
      if (b != 0) {
        entered[count++] = b;
        enterState(ENTER_CODE);
      }
      break;

    case ENTER_CODE:
      if (elapsed >= ENTRY_MS) {
        failedAttempt();
      } else if (b != 0) {
        entered[count++] = b;
        showCode();
        if (count == CODE_LENGTH) {
          if (codeIsCorrect()) {
            failedAttempts = 0;
            lockoutLevel = 0;
            saveCounters();
            enterState(ACCESS_GRANTED);
          } else {
            failedAttempt();
          }
        }
      }
      break;

    case ACCESS_GRANTED:
      if (elapsed >= GRANTED_MS) enterState(IDLE);
      break;

    case ACCESS_DENIED:
      if (elapsed >= DENIED_MS) enterState(IDLE);
      break;

    case LOCKOUT:
      if (elapsed >= lockoutMs) {
        failedAttempts = 0;
        if (lockoutLevel < MAX_LOCKOUT_LEVEL) lockoutLevel++;  // [3] next one is longer
        saveCounters();
        enterState(IDLE);
      } else {
        int secondsLeft = (lockoutMs - elapsed + 999) / 1000;
        if (secondsLeft != lastSecondsShown) {
          lastSecondsShown = secondsLeft;
          lcd.setCursor(0, 1);
          lcd.print("Wait ");
          lcd.print(secondsLeft);
          lcd.print(" s   ");
        }
      }
      break;
```

```
    }
}
```

© 2026 by University of Limerick. Author: Luke Griffin